

Semantic Matching in Medieval Latin: A Benchmark, Layerwise Analysis, and Geometric Repair

Anonymous ACL submission

Abstract

Medieval canon law texts preserve some of the earliest sustained records of judicial reasoning in Western Europe, but they survive in manuscript families shaped by spelling variation, fragmentation, and editorial change. As a result, strict semantic matching is a prerequisite for scholarly workflows such as consolidating duplicate passages and linking fragments across witnesses. We introduce a new Medieval Latin dataset of 1,705 canon law fragments grouped into 840 families of meaning-equivalent variants, and evaluate two tasks: duplicate detection and open-set semantic deduplication. We conduct a layerwise analysis of internal representations across a diverse set of Latin-adapted and multilingual models and uncover a striking failure mode. In Latin-adapted T5 encoders, standard mean-pooled hidden states undergo a pronounced mid-layer collapse, causing cosine similarity to lose discriminative power. In contrast, multilingual encoders such as LaBSE and decoder-based models remain substantially more stable. Motivated by this diagnosis, we apply a simple train-free geometric repair based on All-but-the-Top post-processing, which largely removes the mid-layer failure mode and restores semantic separation. Finally, we show that the resulting representations support more reliable instance-level attribution using integrated gradients, enabling an inspectable, scholar-facing semantic matching pipeline.

1 Introduction

Medieval canon law offers a rich record of early judicial reasoning, reflecting evolving ideas about authority, justice, and social order. Yet, these texts do not exist as stable, single versions. Instead, they are preserved through manuscript traditions that introduce variation in wording, fragmentary transmission, and editorial intervention. As a result, passages that express the same legal principle may

paper_fig_density_2x2.pdf

Figure 1: Cosine similarity distributions for PhilTa at the last layer and at the mid layer, before and after ABTT. At the dip layer. ABTT-based post-processing produces much clearer cosine score separation between semantically similar and different passages.

differ substantially on the surface, while others diverge in subtle but meaningful ways. For scholars, identifying strict semantic equivalence across such variants is a fundamental step in consolidating textual evidence, linking fragments, and tracing the transmission of legal thought.

We study this problem by introducing new data of Medieval Latin canon law passages organized into 840 families of meaning-equivalent variants. We consider two complementary tasks. *(i)* Duplicate detection evaluates whether a pair of passages conveys the same meaning under a strict notion of equivalence. *(ii)* Open-set semantic deduplication reflects corpus-level workflows, requiring models to determine whether a newly observed passage matches any existing passage or represents a previously unseen instance.

A standard approach is to encode passages using pretrained models and compare them with cosine

similarity. While this paradigm is effective for high-resource languages, its behavior in historical languages such as Medieval Latin remains poorly understood. In particular, it remains unclear how semantic information is distributed across model depth and whether commonly used representation extraction methods preserve or distort that information. This motivates a closer examination of model internals rather than treating sentence embeddings as fixed outputs.

To this end, we analyze representations across layers for a range of Latin-adapted and multilingual models. This analysis reveals a consistent failure mode in Latin-adapted T5 encoders: representations obtained via mean pooling lose discriminative structure in intermediate layers, leading to a collapse in semantic separability. Notably, this behavior is not observed to the same extent in multilingual encoders or decoder-based models, which exhibit more stable trends across depth.

This observation points to a geometric explanation. When representations are dominated by a small number of global directions, cosine similarity becomes less sensitive to semantic distinctions, particularly in intermediate layers. We address this issue using a simple, train-free correction based on All-but-the-Top (ABTT) post-processing, which removes dominant components from the embedding space. This correction restores semantic separation and stabilizes performance across layers without modifying model parameters.

A natural approach to both tasks is to embed passages using pretrained models and compare them via cosine similarity. While this paradigm is effective for high-resource languages, its behavior in historical languages such as Medieval Latin remains poorly understood. In particular, standard practice treats sentence representations as fixed outputs, without examining how semantic information is distributed across model depth or how representation extraction affects downstream performance. This raises a key question: to what extent do failures in semantic matching arise from limitations of the underlying model, as opposed to how its internal representations are used?

To answer this, we conduct a layerwise analysis of internal representations across a diverse set of models, including Latin-adapted encoders and multilingual models. We uncover a striking and consistent failure mode. In Latin-adapted T5 encoders, standard mean-pooled hidden states undergo a pronounced mid-layer collapse, in which

cosine similarity loses discriminative power and semantic separation degrades sharply, in some cases approaching chance. In contrast, multilingual encoders such as LaBSE and decoder-based models exhibit substantially more stable behavior across depth. This finding shows that model performance on Medieval Latin semantic matching depends critically on how representations are extracted, and that naïve pooling can obscure usable semantic information.

This observation leads to a key insight: the primary bottleneck is not missing semantic content, but the geometry of the representation space induced by common extraction methods. When representations become anisotropic or dominated by a few global directions, cosine similarity fails to distinguish semantically equivalent from non-equivalent passages, particularly in intermediate layers.

Motivated by this diagnosis, we apply a simple, train-free geometric correction based on All-but-the-Top (ABTT) post-processing (Mu and Viswanath, 2018). By removing dominant directions from the embedding space, this approach restores semantic separation and largely eliminates the mid-layer collapse. The resulting representations are more stable across layers and yield consistently stronger performance on both duplicate detection and open-set semantic deduplication, without requiring any task-specific training.

Beyond improving matching performance, geometric correction also enhances interpretability. We find that applying integrated gradients on corrected representations produces clearer and more localized token-level attributions for semantic matches, whereas raw representations yield diffuse and less informative explanations. This enables an inspectable, scholar-facing workflow in which retrieved matches can be both ranked and explained, supporting more reliable use of NLP tools in humanities research.

In summary, this paper makes the following contributions: (1) we introduce a new benchmark for strict semantic matching in Medieval Latin canon law, including both pairwise and open-set formulations; (2) we provide a layerwise analysis revealing a previously unreported mid-layer collapse in Latin-adapted T5 representations under standard pooling; and (3) we show that a simple train-free geometric correction restores semantic separability across layers and models, and that the corrected representations yield sharper, more localized integrated-

gradient attributions, supporting an inspectable semantic matching pipeline for humanities scholars.

2 Task and Dataset

2.1 Task Definition

We study two related tasks.

Task A: pairwise duplicate detection. Let p_1 and p_2 be two passages and let $\equiv$ denote semantic equivalence, so that $p_1 \equiv p_2$ holds exactly when the two passages express the same meaning. Given an input pair (p_1, p_2) , the task is to predict

$$y = \begin{cases} 1 & \text{if } p_1 \equiv p_2, \\ 0 & \text{otherwise.} \end{cases}$$

The model produces $\cos(e_{p_1}, e_{p_2})$ from its pooled embeddings, with no access to directory structure or any threshold. Task A is evaluated only at the pairwise level on the test $n \times n$ cosine matrix.

Task B: directory routing. Task B operationalizes the same embeddings into the deployment workflow, which does require directory knowledge. Given a query passage q and the existing labelled directories, the system either routes q to one of those directories or flags it as new. We evaluate two variants of this routing decision. (i) *Autonomous routing*. For each query we compute the maximum cosine to any other test file and compare it to a learned threshold τ (fit on the training split via the F_1 -optimal cut on same-directory vs. different-directory pairs). If the score is above τ the query is routed to its top-1 neighbour’s directory; otherwise it is flagged as new. (ii) *Top-k ranking*. The system scores each directory by the maximum cosine similarity between q and any document already assigned to that directory, adds a pseudo-option `__NEW__` with score τ , and presents the top- k options to a human expert; we simulate that expert by assuming the labeled directory would be selected whenever it appears in the shortlist.

2.2 Dataset Construction

This is the part we agreed you’d fill in (professor)

2.3 Train/Test Split

The corpus contains 1,705 medieval Latin manuscript fragments drawn from canon law and organized into 840 directories. Each directory corresponds to one original legal text, and the files inside it are manuscript copies that express the same content. Directory sizes range from 1 to 10 files.

Category	Dirs	Train	Test
Singletons (1 file)	545	272	273
Doubletons (2 files)	108	108	108
Multi-file (≥ 3)	187	467	477
Total	840	847	858

Table 1: Train and test split statistics under the v2 varied-allocation protocol. Doubleton directories are assigned as whole folders (54 to train, 54 to test); all 108 train doubleton files therefore contribute within-directory positive pairs. The test set contains 535 “existing” files, which have a same-directory partner in test, and 323 “new” files, which do not.

We split the corpus into an 847/858 train/test set with a leak-free protocol (Table 1).¹ The 545 singleton directories are distributed roughly evenly between the two splits at the file level. Doubleton directories (108 in total) are assigned as whole folders, 54 to train and 54 to test, so that both files of each doubleton remain together; this contributes 54 within-directory positive training pairs that a file-level doubleton split would discard. For multi-file directories (≥ 3 files, 187 directories), we vary the train allocation across folders within each size class: for a class of K folders with directory size n , we distribute train_n uniformly over $\{1, \dots, n-1\}$ via stratified allocation, with the remainder weighted toward values nearest $n/2$. The class average remains balanced near 50/50, while every size class contributes same-folder positive pairs to training. This avoids the pathological case in which proportional splitting forces every size-3 directory to (1 train, 2 test) and contributes zero same-folder positives from that class. The redesign yields 565 positive training pairs, a roughly 60% increase over the previous proportional split, and a more defensible setup in which no directory size class is structurally excluded from contributing same-folder training positives.

All fitting, including ABTT principal components and threshold tuning, uses training data only. The training set contains 565 positive pairs and the test set contains 595 positive pairs.

3 Methodology

3.1 Models

We evaluate six pre-trained models from three architecture families:

¹Implemented in `src/canon_split_v2.py`; deterministic with seed 42.

- **LaTa**: a T5-based seq2seq model pre-trained on Latin (Riemenschneider and Frank, 2023). We extract encoder hidden states (12 transformer blocks).
- **PhilTa**: a philologically-oriented variant of LaTa, also T5-based (Riemenschneider and Frank, 2023).
- **LaBSE**: a language-agnostic BERT sentence encoder covering 109 languages (Feng et al., 2022) (12 encoder layers).
- **mt5-base**: a multilingual T5 encoder-decoder (Xue et al., 2021) (12 encoder layers).
- **Qwen3-0.6B**: a multilingual decoder-only embedding model (28 layers) (Qwen Team, 2025).
- **KaLM-mini**: a multilingual decoder-only embedding model (24 layers) (Lu et al., 2025).

3.2 Embedding Extraction

We extract hidden-state representations from every transformer block, starting at layer 1 and ending at the final block, while skipping layer 0, the static embedding table. LaTa, PhilTa, LaBSE, and mt5-base each contribute 12 layers. Qwen3-0.6B contributes 28, and KaLM-mini contributes 24. Token representations at each layer are then pooled into a single sentence vector, either by plain mean pooling or by SIF weighting. For the decoder-only models, last-token pooling is another natural choice; we compare it against mean pooling under ABTT post-processing across all six models in Appendix A.

For the paper-facing pooling baseline, we remove only tokenizer artifacts with no lexical content, such as whitespace, padding, or tokenizer-prefix-only tokens. We keep punctuation and numerals in the pooled representations. In the explanation figures, though, we always show the original decoded tokens rather than filtered display tokens.

3.3 Post-Processing: ABTT

Our main post-processing method is All-but-the-Top (ABTT) (Mu and Viswanath, 2018). We center the pooled sentence embeddings, compute the top- D principal components by SVD on the training set, and project those components out of both train and test embeddings. At each layer we search $D \in \{1, 2, 3, 5, 7, 10\}$ and keep the value that maximizes training-set directory accuracy at rank 1 in Task B. We refer to that as the **optimal** setting.

We also compare against two simpler alternatives. One is SIF-weighted pooling (Arora et al., 2017) without ABTT. The other is PCA whitening (Su et al., 2021). SIF assigns each token w_i the weight $\alpha_i = a/(a + p(w_i))$, where $p(w_i)$ is the unigram probability estimated on the training set and $a = 10^{-3}$.

3.4 Example-Level Attribution

For the explanation figures, we use integrated gradients (Sundararajan et al., 2017) as a local attribution method. Given a query passage and one candidate partner passage, we compute per-sequence token attributions toward pair cosine similarity. The displayed axes always use the original decoded tokens rather than filtered display tokens, even when a token’s attribution is close to zero. The heatmap in Figure 6 is therefore a derived pairwise view: token-token cosine similarity is combined with the marginal integrated-gradient scores of the two sequences. We show that view before and after ABTT on the same example. We also render companion self-attention maps for the same pair as supporting context only, since ABTT is applied after encoding and does not change the model’s internal attention weights.

We complement integrated gradients with a retrieval-aligned learned-mask attribution adapted from MaRC (Brinner and Zarrieß, 2023). MaRC was proposed for classification: a soft mask $\mathbf{m} \in [0, 1]^{T_q}$ over input tokens is optimized so that the masked forward preserves the target class logit. We adapt the same construction to retrieval, asking the mask to preserve the pair’s cosine similarity instead.

Let $\mathbf{E}(\mathbf{x}_q) \in \mathbb{R}^{T_q \times d_{\text{emb}}}$ denote the input-embedding lookup, $E_{>0}$ the encoder with the lookup removed, and $\mathbf{z}_c^*$ the ABTT-cleaned mean-pooled candidate, held fixed throughout optimization. We parameterize $\mathbf{m} = \sigma(\boldsymbol{\zeta})$ with free logits $\boldsymbol{\zeta} \in \mathbb{R}^{T_q}$ and write $\mathcal{I}$ for the content-token positions (attention-active, not tokenizer artifacts). The objective is

$$\mathcal{L}(\mathbf{m}) = (\cos(\text{ABTT}(\tilde{\mathbf{z}}_q(\mathbf{m})), \mathbf{z}_c^*) - \cos(\text{ABTT}(\mathbf{z}_q), \mathbf{z}_c^*))^2 + \lambda \sum_{i \in \mathcal{I}} \quad (1)$$

where $\tilde{\mathbf{z}}_q(\mathbf{m}) = \text{pool}(E_{>0}(\text{diag}(\mathbf{m}) \mathbf{E}(\mathbf{x}_q)), \mathbf{a}_q)$ is the masked pooled query, (λ, γ) control sparsity and total-variation smoothness, and logits outside $\mathcal{I}$ are frozen at $-\infty$. We run 200 Adam steps per pair, initialize $\zeta_0 = \log 9$ so that $\mathbf{m} \approx 0.9$ at

the start, and report both a baseline variant that uses raw cosines in place of the ABTT-cleaned cosines in Equation 1 and the ABTT variant stated above. The candidate-side mask is obtained by swapping the roles of query and candidate. We use this method alongside integrated gradients in the example-level analysis; quantitative faithfulness comparisons against other attribution methods are left as accompanying work.

3.5 Evaluation Metrics

Task A. Both Task A metrics are properties of the test $n \times n$ pairwise cosine matrix. **AUROC** treats every same-directory test pair as a positive and every different-directory pair as a negative, summarising how well cosine similarity ranks positives above negatives. **Cosine gap** is the difference between the mean same-directory and mean different-directory cosine, a direct measure of class separation. Neither uses a threshold and neither requires the model to know anything about directories.

Task B. Task B has two sets of metrics, one for each variant. For *autonomous routing*, we report **overall assignment accuracy**: the rate at which the binary existing-vs-new decision matches ground truth, plus a correct directory match on files predicted existing. We decompose it into **existing accuracy** (restricted to files that have a same-directory partner in the test set) and **new accuracy** (restricted to files that should have been flagged as novel). For *top-k ranking*, we report **cumulative top- K accuracy**: for each K from 1 to 5, the fraction of test queries whose labeled directory appears among the model’s top K options. We also report **Acc@1** averaged over $M = 5$ query/reference reseeds as a stable per-layer ranking signal. Training-set directory accuracy at rank 1 is the model-selection signal for ABTT’s D .

4 Results

4.1 Task A: Pairwise Duplicate Detection

Table 2: Per-layer Task A pairwise duplicate-detection metrics for each model, comparing baseline (mean pooling, no correction) against `sif_abtt_optimal` (SIF weighting plus ABTT with D tuned per layer on the train split). Both metrics are computed over the test $n \times n$ cosine matrix with no threshold and no directory routing: AUROC treats same-directory test pairs as positives, and cosine gap is the difference between mean same-directory and mean different-directory cosine. Rows in bold mark the best layer per model, selected by ABTT AUROC.

Model	Layer	Task A: Pairwise Duplicate Detection			
		AUROC (base)	AUROC (ABTT)	Cosine gap (base)	Cosine gap (ABTT)
LaTa	1	0.933	0.974	0.286	0.590
	2	0.565	0.970	0.020	0.580
	3	0.515	0.964	-0.046	0.580
	4	0.502	0.968	-0.065	0.586
	5	0.498	0.969	-0.068	0.584
	6	0.497	0.964	-0.067	0.580
	7	0.499	0.965	-0.070	0.579
	8	0.504	0.964	-0.073	0.576
	9	0.504	0.964	-0.071	0.579
	10	0.507	0.965	-0.070	0.576
	11	0.510	0.965	-0.065	0.579
	12	0.938	0.963	0.238	0.561
PhilTa	1	0.939	0.973	0.231	0.572
	2	0.911	0.970	0.161	0.565
	3	0.601	0.966	0.085	0.558
	4	0.551	0.970	0.064	0.563
	5	0.543	0.976	0.054	0.574
	6	0.541	0.980	0.048	0.567
	7	0.542	0.980	0.047	0.570
	8	0.543	0.981	0.047	0.573
	9	0.539	0.977	0.042	0.563
	10	0.539	0.977	0.040	0.556
	11	0.540	0.978	0.041	0.542
	12	0.911	0.968	0.181	0.517
LaBSE	1	0.807	0.974	0.060	0.495
	2	0.835	0.974	0.038	0.514
	3	0.855	0.968	0.063	0.517
	4	0.864	0.966	0.074	0.511
	5	0.866	0.963	0.066	0.500
	6	0.895	0.964	0.078	0.525
	7	0.889	0.961	0.089	0.516
	8	0.881	0.966	0.080	0.521
	9	0.918	0.971	0.091	0.558
	10	0.936	0.974	0.103	0.579
	11	0.961	0.980	0.061	0.609
	12	0.957	0.982	0.183	0.595
Qwen3-0.6B	1	0.859	0.980	0.033	0.552
	2	0.864	0.972	0.024	0.540
	3	0.864	0.975	0.037	0.549
	4	0.861	0.977	0.036	0.543
	5	0.862	0.971	0.030	0.528
	6	0.868	0.974	0.035	0.537
	7	0.877	0.971	0.033	0.538
	8	0.884	0.969	0.037	0.531
	9	0.882	0.968	0.034	0.528
	10	0.888	0.966	0.031	0.521
	11	0.896	0.964	0.030	0.516
	12	0.903	0.967	0.027	0.528
	13	0.908	0.967	0.026	0.524
	14	0.906	0.966	0.024	0.516
	15	0.902	0.967	0.022	0.513
	16	0.911	0.965	0.026	0.508
	17	0.917	0.974	0.032	0.519
	18	0.934	0.972	0.032	0.526
	19	0.937	0.976	0.035	0.530
	20	0.943	0.971	0.041	0.544

(continued on next page)

Task A: Pairwise Duplicate Detection					
Model	Layer	AUROC (base)	AUROC (ABTT)	Cosine gap (base)	Cosine gap (ABTT)
	21	0.953	0.971	0.047	0.548
	22	0.963	0.973	0.044	0.559
	23	0.960	0.973	0.038	0.557
	24	0.962	0.973	0.032	0.564
	25	0.960	0.974	0.027	0.563
	26	0.965	0.979	0.024	0.560
	27	0.966	0.977	0.017	0.554
	28	0.958	0.967	0.098	0.550
KaLM-mini	1	0.888	0.979	0.064	0.544
	2	0.908	0.981	0.044	0.535
	3	0.882	0.977	0.062	0.536
	4	0.872	0.974	0.060	0.535
	5	0.862	0.971	0.062	0.527
	6	0.873	0.966	0.062	0.518
	7	0.889	0.971	0.072	0.522
	8	0.891	0.975	0.072	0.525
	9	0.893	0.966	0.066	0.505
	10	0.893	0.963	0.067	0.496
	11	0.895	0.968	0.061	0.483
	12	0.905	0.965	0.071	0.487
	13	0.901	0.963	0.066	0.476
	14	0.904	0.968	0.063	0.477
	15	0.907	0.968	0.067	0.480
	16	0.917	0.966	0.071	0.492
	17	0.926	0.968	0.075	0.514
	18	0.944	0.976	0.060	0.530
	19	0.955	0.978	0.061	0.538
	20	0.959	0.978	0.054	0.542
	21	0.961	0.982	0.043	0.562
	22	0.969	0.982	0.050	0.583
	23	0.970	0.984	0.056	0.579
	24	0.954	0.975	0.243	0.521
mT5-base	1	0.823	0.971	0.027	0.519
	2	0.823	0.964	0.026	0.511
	3	0.805	0.956	0.025	0.506
	4	0.800	0.966	0.040	0.505
	5	0.654	0.972	0.218	0.484
	6	0.657	0.970	0.238	0.486
	7	0.657	0.966	0.243	0.480
	8	0.656	0.967	0.248	0.466
	9	0.660	0.966	0.235	0.469
	10	0.659	0.964	0.225	0.455
	11	0.660	0.964	0.213	0.448
	12	0.839	0.968	0.082	0.446

Table 2 reports per-layer Task A pairwise metrics for each model, comparing the uncorrected baseline (mean pooling) against `sif_abtt_optimal` (SIF weighting plus ABTT with D tuned per layer on the train split). Both AUROC and cosine gap are read off the test $n \times n$ cosine matrix without any threshold. Bolded rows mark each model’s best layer by ABTT AUROC. The two columns drift apart sharply in the middle layers of the Latin T5 encoders (LaTa, PhilTa) and in `mt5-base`, and much less in LaBSE, Qwen3-0.6B, and KaLM-mini. The remainder of this section unpacks where these numbers come from layer by layer.

4.2 The Mid-Layer Dip

The main result shows up immediately in the layerwise plots. Baseline mean-pooled embeddings from LaTa and PhilTa collapse in the middle encoder layers. LaTa drops from strong early-layer AUCROC to below 0.5, which is chance, and PhilTa follows almost the same path. Both recover somewhat by the final layer.

LaBSE and `mt5-base` do not show the same kind of collapse, and even their weakest layers remain well above chance. Qwen3-0.6B and KaLM-mini also vary much less across depth, which fits their decoder-only architecture.

After ABTT, the scores within each model cluster much more tightly across depth.

4.3 Similarity Distributions

Figure 1 shows the same pattern from another angle. It compares PhilTa at the last layer and at its dip layer. Before ABTT, the dip-layer cosine scores for true duplicates and non-duplicates overlap heavily. After ABTT, those distributions separate much more clearly at the same layer.

4.4 Cosine Gap Analysis

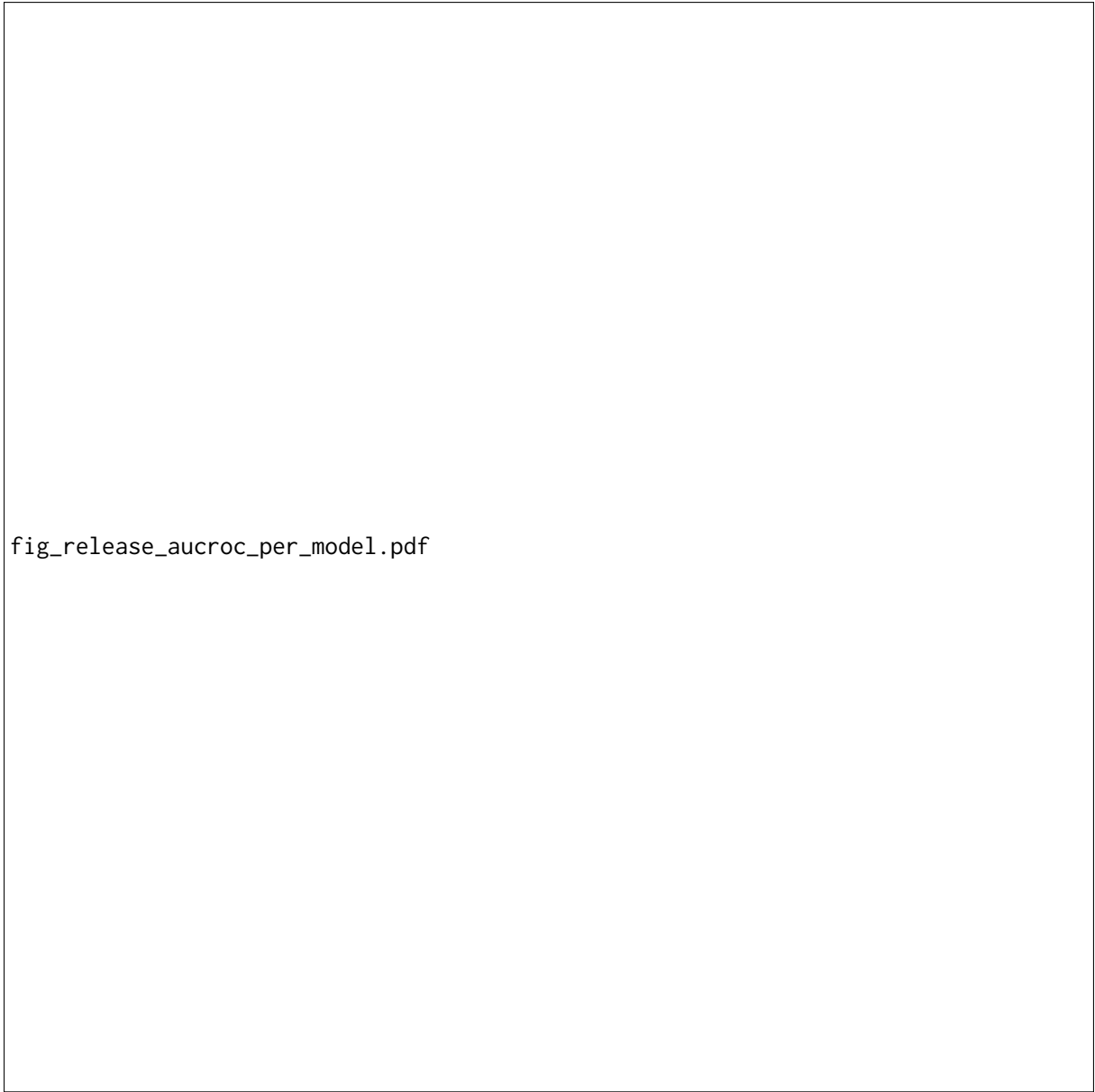
The cosine-gap plots tell the same story in a more compact way. In LaTa and PhilTa, the baseline gap nearly disappears in the dip region. ABTT restores a clear positive gap across all layers.

4.5 Cluster Geometry

Cosine-gap and AUCROC curves summarize separability in aggregate. Two-dimensional projections make the geometric effect of ABTT visible directly. Figures 4 and 5 show t-SNE (van der Maaten and Hinton, 2008) and UMAP (McInnes et al., 2018) embeddings of the 1,705 labelled fragments at each

of three representative models’ best layer, comparing mean-pooled baseline to pure ABTT (no SIF, $D=10$). To isolate the geometric signal, we plot only the six most-populated directories (sizes 8–10); the remaining winnable fragments and singletons are suppressed. Under baseline, colored points from different directories often bleed into one another. Under ABTT, same-color fragments consistently collapse onto tight, well-separated clusters; the effect is strongest on `mt5-base`, where baseline silhouette is negative on both projections and ABTT brings it above 0.2. The pattern holds across both projection algorithms, indicating that the tightening reflects the post-processed embedding geometry rather than an artifact of either dimensionality-reduction method. The remaining three models (LaBSE, Qwen3-0.6B, KaLM-mini) show the same pattern and are reported in Appendix B.

4.6 Task B: Directory Routing



fig_release_aucroc_per_model.pdf

Figure 2: AUCROC across normalized layer depth for the six models. LaTa and PhilTa show the clearest mid-layer dip before post-processing. ABTT flattens those curves much more than SIF or whitening.

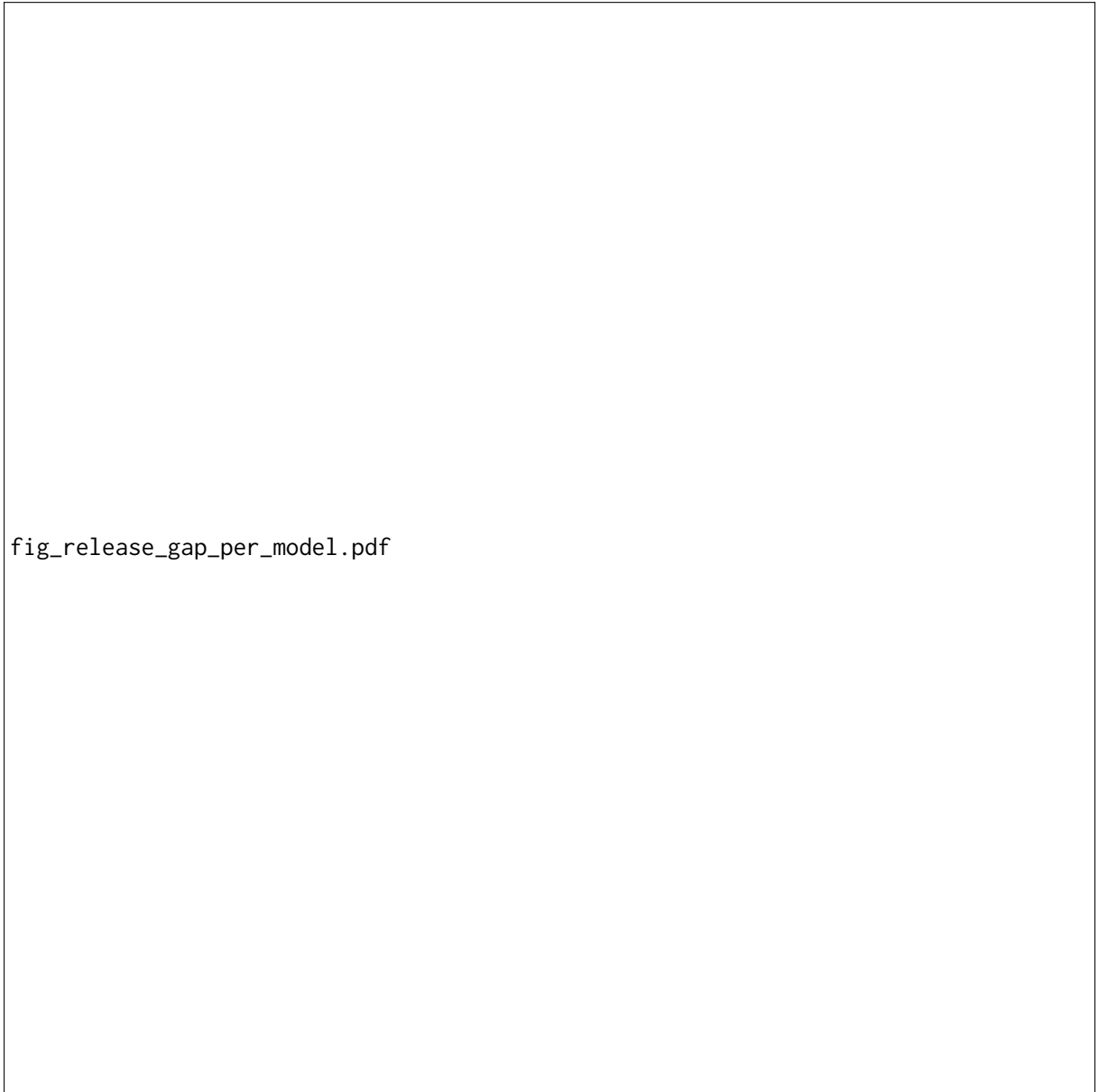
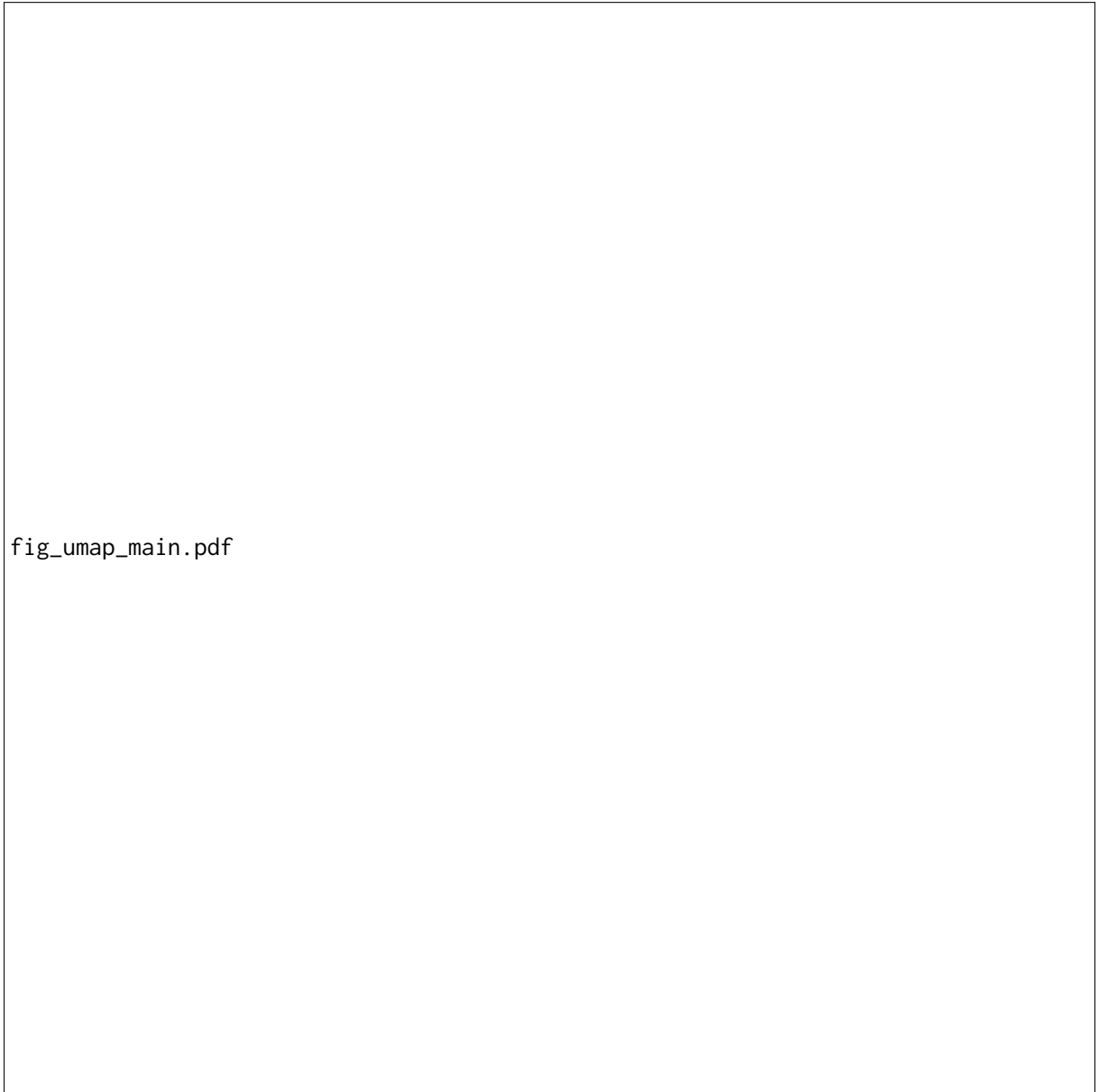


Figure 3: Cosine gap across normalized layer depth for the six models. In the Latin T5 models, the baseline gap nearly vanishes in the middle layers. ABTT restores a positive gap across depth more reliably than the alternative baselines.



Figure 4: t-SNE projections of the 1,705 labelled fragments at each model’s best layer (rows: LaTa, PhilTa, mT5-base), for mean-pooled baseline and pure ABTT ($D=10$) columns. Colors mark the six most-populated directories (CNEO.315.13, CNIC.325.16, CANC.314.4, CANC.314.20, CNIC.325.6, CNIC.325.4; sizes 8–10); all other fragments are suppressed. ABTT produces visibly tighter same-color clusters than the baseline, with winnable-fragment silhouette scores rising by 0.09–0.75 on every model.



fig_umap_main.pdf

Figure 5: UMAP projections, same configuration as Figure 4. The per-method separation pattern is reproduced under a different dimensionality-reduction algorithm, supporting that the cluster-tightening is a property of the embedding geometry rather than of the projection method.

Table 3: Per-layer Task B autonomous routing accuracy for each model. For every test file we compute $s_i = \max_{j \neq i} \cos(e_i, e_j)$ against the rest of the test set and compare to the learned threshold τ (fit on train via F_1 -optimal cut on same-directory vs. different-directory pairs). The decision is binary first (existing if $s_i \geq \tau$, else new); only files predicted existing are then routed to their top-1 neighbour’s directory. We report **existing accuracy** (restricted to files whose true partner is in the test set), **new accuracy** (files that should be flagged as novel), and **overall assignment accuracy**. Rows in bold mark the best layer per model, selected by ABTT overall assignment accuracy.

Model	Layer	Task B: Autonomous Routing (file-level, τ -thresholded)					
		Existing (base)	Existing (ABTT)	New (base)	New (ABTT)	Overall (base)	Overall (ABTT)
LaTa	1	0.630	0.899	0.916	0.898	0.738	0.899
	2	0.153	0.899	0.913	0.901	0.439	0.900
	3	0.256	0.907	0.842	0.907	0.477	0.907
	4	0.389	0.920	0.718	0.873	0.513	0.902
	5	0.374	0.914	0.715	0.892	0.502	0.906

(continued on next page)

Task B: Autonomous Routing (file-level, τ -thresholded)							
Model	Layer	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)	Overall (base)	Overall (ABTT)
	6	0.378	0.910	0.712	0.870	0.503	0.895
	7	0.372	0.890	0.715	0.926	0.501	0.903
	8	0.379	0.877	0.706	0.916	0.502	0.892
	9	0.366	0.888	0.724	0.892	0.501	0.889
	10	0.344	0.884	0.752	0.901	0.498	0.890
	11	0.310	0.907	0.786	0.836	0.490	0.880
	12	0.576	0.880	0.932	0.938	0.710	0.902
	1	0.591	0.905	0.904	0.929	0.709	0.914
	2	0.479	0.921	0.867	0.842	0.625	0.892
	3	0.193	0.899	0.889	0.879	0.455	0.892
	4	0.148	0.903	0.923	0.858	0.439	0.886
	5	0.252	0.905	0.796	0.885	0.457	0.897
PhilTa	6	0.338	0.908	0.700	0.901	0.474	0.906
	7	0.301	0.903	0.746	0.910	0.469	0.906
	8	0.262	0.897	0.808	0.867	0.467	0.886
	9	0.303	0.905	0.762	0.879	0.476	0.895
	10	0.299	0.899	0.780	0.870	0.480	0.888
	11	0.256	0.897	0.824	0.867	0.470	0.886
	12	0.338	0.908	0.898	0.882	0.549	0.899
	1	0.222	0.897	0.898	0.867	0.477	0.886
	2	0.381	0.880	0.811	0.867	0.543	0.875
	3	0.277	0.912	0.907	0.774	0.514	0.860
	4	0.372	0.907	0.885	0.808	0.565	0.869
	5	0.327	0.854	0.901	0.861	0.543	0.857
LaBSE	6	0.548	0.875	0.882	0.867	0.674	0.872
	7	0.398	0.869	0.892	0.839	0.584	0.858
	8	0.385	0.865	0.870	0.836	0.568	0.854
	9	0.421	0.886	0.929	0.814	0.612	0.859
	10	0.606	0.884	0.892	0.830	0.713	0.864
	11	0.785	0.929	0.913	0.842	0.833	0.896
	12	0.729	0.895	0.938	0.854	0.808	0.880
	1	0.264	0.890	0.960	0.941	0.526	0.909
	2	0.437	0.880	0.814	0.916	0.579	0.894
	3	0.250	0.884	0.947	0.929	0.513	0.901
	4	0.411	0.903	0.848	0.907	0.576	0.904
Qwen3-0.6B	5	0.181	0.892	0.932	0.935	0.464	0.908
	6	0.422	0.907	0.830	0.904	0.576	0.906
	7	0.250	0.910	0.947	0.913	0.513	0.911
	8	0.409	0.910	0.895	0.913	0.592	0.911
	9	0.460	0.925	0.851	0.885	0.607	0.910
	10	0.533	0.899	0.836	0.901	0.647	0.900
	11	0.293	0.897	0.978	0.885	0.551	0.893
	12	0.407	0.897	0.938	0.904	0.607	0.900
	13	0.432	0.908	0.944	0.879	0.625	0.897
	14	0.456	0.910	0.926	0.864	0.633	0.893
	15	0.469	0.910	0.920	0.864	0.639	0.893
	16	0.639	0.899	0.830	0.848	0.711	0.880
	17	0.480	0.867	0.935	0.892	0.652	0.876
	18	0.553	0.892	0.944	0.867	0.700	0.882
	19	0.514	0.821	0.966	0.898	0.684	0.850
	20	0.654	0.912	0.898	0.833	0.746	0.882
	21	0.647	0.890	0.950	0.895	0.761	0.892
	22	0.731	0.914	0.938	0.858	0.809	0.893
	23	0.622	0.908	0.966	0.867	0.752	0.893
	24	0.748	0.908	0.935	0.882	0.818	0.899
	25	0.574	0.910	0.975	0.864	0.725	0.893
	26	0.613	0.901	0.975	0.907	0.749	0.903
	27	0.793	0.864	0.833	0.957	0.808	0.899
	28	0.796	0.884	0.867	0.901	0.823	0.890
KaLM-mini	1	0.391	0.907	0.907	0.910	0.585	0.908
	2	0.493	0.903	0.848	0.916	0.627	0.908
	3	0.443	0.899	0.870	0.920	0.604	0.907

(continued on next page)

Task B: Autonomous Routing (file-level, τ -thresholded)							
Model	Layer	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)	Overall (base)	Overall (ABTT)
	4	0.308	0.897	0.935	0.907	0.544	0.901
	5	0.338	0.892	0.882	0.907	0.543	0.897
	6	0.256	0.905	0.910	0.892	0.502	0.900
	7	0.475	0.895	0.854	0.910	0.618	0.901
	8	0.467	0.862	0.870	0.904	0.619	0.878
	9	0.469	0.873	0.885	0.882	0.626	0.876
	10	0.449	0.828	0.885	0.929	0.613	0.866
	11	0.458	0.849	0.885	0.916	0.619	0.874
	12	0.507	0.854	0.895	0.907	0.653	0.874
	13	0.540	0.843	0.895	0.898	0.674	0.864
	14	0.593	0.852	0.845	0.876	0.688	0.861
	15	0.566	0.864	0.898	0.870	0.691	0.866
	16	0.550	0.871	0.935	0.839	0.695	0.859
	17	0.675	0.888	0.920	0.873	0.767	0.882
	18	0.697	0.880	0.960	0.876	0.796	0.879
	19	0.721	0.877	0.966	0.879	0.814	0.878
	20	0.705	0.873	0.966	0.882	0.803	0.876
	21	0.727	0.852	0.966	0.916	0.817	0.876
	22	0.850	0.852	0.916	0.916	0.875	0.876
	23	0.849	0.873	0.920	0.895	0.875	0.881
	24	0.787	0.871	0.944	0.867	0.846	0.869
mT5-base	1	0.135	0.908	0.981	0.904	0.453	0.907
	2	0.110	0.908	0.975	0.895	0.436	0.903
	3	0.161	0.892	0.926	0.885	0.449	0.889
	4	0.269	0.824	0.858	0.910	0.491	0.857
	5	0.422	0.770	0.659	0.920	0.512	0.826
	6	0.636	0.832	0.489	0.793	0.580	0.817
	7	0.594	0.819	0.517	0.854	0.565	0.832
	8	0.557	0.843	0.560	0.820	0.558	0.834
	9	0.364	0.845	0.746	0.845	0.508	0.845
	10	0.314	0.832	0.783	0.833	0.491	0.832
	11	0.297	0.791	0.780	0.858	0.479	0.816
	12	0.198	0.841	0.926	0.885	0.472	0.858

Table 4: Per-layer Task B accuracy for each model, averaged over $M = 5$ query/reference reseeds of the v2 train/test split (mean $\pm$ std). baseline is mean pooling without correction; `sif_abtt_optimal` applies SIF weighting plus ABTT with D tuned per layer on the train split. Overall assignment accuracy coincides with Acc@1 in the Task B mseed pipeline (assignment = top-1 pick), so we instead report the existing vs. new decomposition. Rows in bold mark the best layer per model, selected by ABTT Acc@1. See Table 3 for the complementary autonomous-routing view.

Task B: Top-K Ranking (5-seed mean $\pm$ std)							
Model	Layer	Acc@1 (base)	Acc@1 (ABTT)	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)
LaTa	1	0.731 $\pm$ 0.009	0.889 $\pm$ 0.009	0.562 $\pm$ 0.004	0.842 $\pm$ 0.009	0.949 $\pm$ 0.011	0.950 $\pm$ 0.011
	2	0.428 $\pm$ 0.011	0.887 $\pm$ 0.005	0.048 $\pm$ 0.006	0.840 $\pm$ 0.005	0.920 $\pm$ 0.008	0.949 $\pm$ 0.009
	3	0.392 $\pm$ 0.012	0.889 $\pm$ 0.008	0.046 $\pm$ 0.008	0.845 $\pm$ 0.006	0.840 $\pm$ 0.020	0.946 $\pm$ 0.013
	4	0.330 $\pm$ 0.018	0.888 $\pm$ 0.008	0.047 $\pm$ 0.008	0.858 $\pm$ 0.007	0.696 $\pm$ 0.030	0.927 $\pm$ 0.013
	5	0.326 $\pm$ 0.020	0.890 $\pm$ 0.008	0.039 $\pm$ 0.004	0.856 $\pm$ 0.005	0.698 $\pm$ 0.028	0.935 $\pm$ 0.014
	6	0.321 $\pm$ 0.018	0.890 $\pm$ 0.008	0.038 $\pm$ 0.007	0.861 $\pm$ 0.008	0.688 $\pm$ 0.026	0.927 $\pm$ 0.014
	7	0.327 $\pm$ 0.020	0.895 $\pm$ 0.006	0.044 $\pm$ 0.007	0.846 $\pm$ 0.006	0.694 $\pm$ 0.029	0.957 $\pm$ 0.010
	8	0.325 $\pm$ 0.022	0.883 $\pm$ 0.007	0.046 $\pm$ 0.010	0.832 $\pm$ 0.006	0.685 $\pm$ 0.032	0.949 $\pm$ 0.011
	9	0.331 $\pm$ 0.019	0.885 $\pm$ 0.008	0.045 $\pm$ 0.010	0.846 $\pm$ 0.006	0.701 $\pm$ 0.024	0.935 $\pm$ 0.013
	10	0.346 $\pm$ 0.017	0.888 $\pm$ 0.002	0.044 $\pm$ 0.010	0.844 $\pm$ 0.007	0.736 $\pm$ 0.022	0.944 $\pm$ 0.010
	11	0.360 $\pm$ 0.018	0.881 $\pm$ 0.006	0.045 $\pm$ 0.009	0.860 $\pm$ 0.008	0.767 $\pm$ 0.028	0.907 $\pm$ 0.011
	12	0.707 $\pm$ 0.015	0.889 $\pm$ 0.005	0.511 $\pm$ 0.017	0.834 $\pm$ 0.005	0.961 $\pm$ 0.008	0.961 $\pm$ 0.010
PhilTa	1	0.706 $\pm$ 0.016	0.898 $\pm$ 0.007	0.532 $\pm$ 0.018	0.856 $\pm$ 0.009	0.931 $\pm$ 0.012	0.953 $\pm$ 0.011
	2	0.625 $\pm$ 0.018	0.889 $\pm$ 0.009	0.408 $\pm$ 0.016	0.866 $\pm$ 0.009	0.905 $\pm$ 0.017	0.919 $\pm$ 0.017
	3	0.439 $\pm$ 0.013	0.892 $\pm$ 0.006	0.071 $\pm$ 0.014	0.856 $\pm$ 0.008	0.917 $\pm$ 0.013	0.939 $\pm$ 0.009

(continued on next page)

Task B: Top-K Ranking (5-seed mean $\pm$ std)							
Model	Layer	Acc@1 (base)	Acc@1 (ABTT)	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)
	4	0.420 $\pm$ 0.016	0.893 $\pm$ 0.009	0.027 $\pm$ 0.007	0.858 $\pm$ 0.006	0.928 $\pm$ 0.015	0.938 $\pm$ 0.015
	5	0.374 $\pm$ 0.015	0.896 $\pm$ 0.010	0.042 $\pm$ 0.011	0.872 $\pm$ 0.007	0.805 $\pm$ 0.018	0.925 $\pm$ 0.014
	6	0.338 $\pm$ 0.015	0.897 $\pm$ 0.008	0.059 $\pm$ 0.010	0.861 $\pm$ 0.007	0.698 $\pm$ 0.025	0.945 $\pm$ 0.014
	7	0.361 $\pm$ 0.016	0.900 $\pm$ 0.009	0.056 $\pm$ 0.010	0.859 $\pm$ 0.005	0.757 $\pm$ 0.021	0.952 $\pm$ 0.016
	8	0.390 $\pm$ 0.016	0.891 $\pm$ 0.010	0.058 $\pm$ 0.012	0.858 $\pm$ 0.007	0.820 $\pm$ 0.017	0.933 $\pm$ 0.018
	9	0.372 $\pm$ 0.015	0.894 $\pm$ 0.006	0.066 $\pm$ 0.012	0.861 $\pm$ 0.006	0.768 $\pm$ 0.026	0.936 $\pm$ 0.011
	10	0.377 $\pm$ 0.013	0.892 $\pm$ 0.006	0.066 $\pm$ 0.012	0.855 $\pm$ 0.003	0.781 $\pm$ 0.021	0.941 $\pm$ 0.015
	11	0.394 $\pm$ 0.013	0.889 $\pm$ 0.006	0.058 $\pm$ 0.008	0.851 $\pm$ 0.008	0.829 $\pm$ 0.018	0.937 $\pm$ 0.008
	12	0.563 $\pm$ 0.010	0.896 $\pm$ 0.007	0.279 $\pm$ 0.011	0.863 $\pm$ 0.009	0.932 $\pm$ 0.005	0.938 $\pm$ 0.014
LaBSE	1	0.501 $\pm$ 0.011	0.858 $\pm$ 0.008	0.170 $\pm$ 0.002	0.838 $\pm$ 0.007	0.930 $\pm$ 0.014	0.884 $\pm$ 0.010
	2	0.522 $\pm$ 0.010	0.861 $\pm$ 0.006	0.272 $\pm$ 0.003	0.817 $\pm$ 0.004	0.844 $\pm$ 0.022	0.917 $\pm$ 0.013
	3	0.535 $\pm$ 0.008	0.845 $\pm$ 0.005	0.217 $\pm$ 0.007	0.831 $\pm$ 0.010	0.946 $\pm$ 0.007	0.864 $\pm$ 0.009
	4	0.568 $\pm$ 0.008	0.849 $\pm$ 0.007	0.298 $\pm$ 0.006	0.837 $\pm$ 0.008	0.918 $\pm$ 0.014	0.865 $\pm$ 0.015
	5	0.550 $\pm$ 0.011	0.845 $\pm$ 0.007	0.254 $\pm$ 0.011	0.794 $\pm$ 0.006	0.933 $\pm$ 0.013	0.911 $\pm$ 0.015
	6	0.660 $\pm$ 0.012	0.858 $\pm$ 0.003	0.460 $\pm$ 0.021	0.816 $\pm$ 0.009	0.918 $\pm$ 0.018	0.912 $\pm$ 0.016
	7	0.595 $\pm$ 0.010	0.850 $\pm$ 0.003	0.333 $\pm$ 0.019	0.808 $\pm$ 0.006	0.934 $\pm$ 0.015	0.904 $\pm$ 0.010
	8	0.571 $\pm$ 0.013	0.848 $\pm$ 0.004	0.313 $\pm$ 0.019	0.802 $\pm$ 0.011	0.905 $\pm$ 0.018	0.906 $\pm$ 0.007
	9	0.630 $\pm$ 0.008	0.855 $\pm$ 0.004	0.380 $\pm$ 0.018	0.828 $\pm$ 0.005	0.954 $\pm$ 0.008	0.890 $\pm$ 0.007
	10	0.711 $\pm$ 0.015	0.856 $\pm$ 0.003	0.550 $\pm$ 0.018	0.835 $\pm$ 0.005	0.920 $\pm$ 0.016	0.884 $\pm$ 0.004
	11	0.820 $\pm$ 0.009	0.904 $\pm$ 0.008	0.732 $\pm$ 0.011	0.873 $\pm$ 0.010	0.935 $\pm$ 0.013	0.944 $\pm$ 0.017
	12	0.792 $\pm$ 0.013	0.883 $\pm$ 0.006	0.668 $\pm$ 0.016	0.849 $\pm$ 0.009	0.952 $\pm$ 0.013	0.928 $\pm$ 0.014
Qwen3-0.6B	1	0.549 $\pm$ 0.006	0.892 $\pm$ 0.004	0.221 $\pm$ 0.009	0.865 $\pm$ 0.008	0.972 $\pm$ 0.003	0.927 $\pm$ 0.007
	2	0.570 $\pm$ 0.013	0.882 $\pm$ 0.003	0.362 $\pm$ 0.012	0.828 $\pm$ 0.007	0.839 $\pm$ 0.028	0.951 $\pm$ 0.006
	3	0.540 $\pm$ 0.006	0.889 $\pm$ 0.005	0.214 $\pm$ 0.010	0.827 $\pm$ 0.011	0.962 $\pm$ 0.007	0.969 $\pm$ 0.011
	4	0.581 $\pm$ 0.010	0.891 $\pm$ 0.004	0.354 $\pm$ 0.008	0.844 $\pm$ 0.009	0.874 $\pm$ 0.023	0.952 $\pm$ 0.010
	5	0.486 $\pm$ 0.016	0.894 $\pm$ 0.007	0.133 $\pm$ 0.013	0.839 $\pm$ 0.011	0.943 $\pm$ 0.012	0.965 $\pm$ 0.010
	6	0.566 $\pm$ 0.011	0.897 $\pm$ 0.005	0.349 $\pm$ 0.009	0.857 $\pm$ 0.010	0.848 $\pm$ 0.020	0.949 $\pm$ 0.013
	7	0.540 $\pm$ 0.007	0.902 $\pm$ 0.006	0.211 $\pm$ 0.011	0.862 $\pm$ 0.009	0.967 $\pm$ 0.012	0.955 $\pm$ 0.015
	8	0.608 $\pm$ 0.010	0.903 $\pm$ 0.006	0.361 $\pm$ 0.016	0.861 $\pm$ 0.011	0.927 $\pm$ 0.015	0.958 $\pm$ 0.009
	9	0.610 $\pm$ 0.011	0.901 $\pm$ 0.005	0.391 $\pm$ 0.015	0.867 $\pm$ 0.009	0.892 $\pm$ 0.019	0.944 $\pm$ 0.014
	10	0.636 $\pm$ 0.012	0.890 $\pm$ 0.008	0.450 $\pm$ 0.009	0.847 $\pm$ 0.014	0.877 $\pm$ 0.020	0.946 $\pm$ 0.010
	11	0.577 $\pm$ 0.006	0.886 $\pm$ 0.005	0.257 $\pm$ 0.013	0.838 $\pm$ 0.016	0.991 $\pm$ 0.005	0.948 $\pm$ 0.015
	12	0.626 $\pm$ 0.009	0.893 $\pm$ 0.004	0.365 $\pm$ 0.019	0.844 $\pm$ 0.013	0.964 $\pm$ 0.013	0.958 $\pm$ 0.010
	13	0.638 $\pm$ 0.010	0.888 $\pm$ 0.006	0.382 $\pm$ 0.016	0.849 $\pm$ 0.013	0.970 $\pm$ 0.015	0.938 $\pm$ 0.009
	14	0.644 $\pm$ 0.010	0.885 $\pm$ 0.007	0.401 $\pm$ 0.019	0.852 $\pm$ 0.012	0.958 $\pm$ 0.015	0.929 $\pm$ 0.008
	15	0.646 $\pm$ 0.013	0.885 $\pm$ 0.005	0.409 $\pm$ 0.020	0.852 $\pm$ 0.012	0.952 $\pm$ 0.016	0.929 $\pm$ 0.017
	16	0.689 $\pm$ 0.015	0.873 $\pm$ 0.006	0.544 $\pm$ 0.017	0.837 $\pm$ 0.010	0.877 $\pm$ 0.016	0.920 $\pm$ 0.020
	17	0.657 $\pm$ 0.010	0.861 $\pm$ 0.008	0.419 $\pm$ 0.020	0.807 $\pm$ 0.014	0.964 $\pm$ 0.012	0.932 $\pm$ 0.017
	18	0.708 $\pm$ 0.011	0.868 $\pm$ 0.004	0.503 $\pm$ 0.019	0.833 $\pm$ 0.016	0.973 $\pm$ 0.005	0.913 $\pm$ 0.020
	19	0.690 $\pm$ 0.011	0.873 $\pm$ 0.005	0.462 $\pm$ 0.018	0.859 $\pm$ 0.008	0.986 $\pm$ 0.006	0.892 $\pm$ 0.015
	20	0.742 $\pm$ 0.019	0.880 $\pm$ 0.006	0.588 $\pm$ 0.023	0.858 $\pm$ 0.009	0.941 $\pm$ 0.017	0.909 $\pm$ 0.016
	21	0.757 $\pm$ 0.016	0.878 $\pm$ 0.006	0.584 $\pm$ 0.021	0.835 $\pm$ 0.009	0.981 $\pm$ 0.010	0.934 $\pm$ 0.013
	22	0.798 $\pm$ 0.014	0.882 $\pm$ 0.006	0.670 $\pm$ 0.021	0.854 $\pm$ 0.004	0.965 $\pm$ 0.006	0.917 $\pm$ 0.017
	23	0.757 $\pm$ 0.013	0.888 $\pm$ 0.007	0.578 $\pm$ 0.016	0.857 $\pm$ 0.005	0.989 $\pm$ 0.004	0.929 $\pm$ 0.016
	24	0.810 $\pm$ 0.014	0.894 $\pm$ 0.005	0.694 $\pm$ 0.022	0.861 $\pm$ 0.005	0.960 $\pm$ 0.003	0.936 $\pm$ 0.015
	25	0.732 $\pm$ 0.012	0.890 $\pm$ 0.009	0.528 $\pm$ 0.016	0.866 $\pm$ 0.005	0.997 $\pm$ 0.003	0.922 $\pm$ 0.017
	26	0.749 $\pm$ 0.013	0.895 $\pm$ 0.007	0.561 $\pm$ 0.017	0.858 $\pm$ 0.002	0.991 $\pm$ 0.003	0.943 $\pm$ 0.017
	27	0.811 $\pm$ 0.004	0.881 $\pm$ 0.003	0.741 $\pm$ 0.008	0.824 $\pm$ 0.007	0.901 $\pm$ 0.011	0.954 $\pm$ 0.011
	28	0.822 $\pm$ 0.010	0.870 $\pm$ 0.007	0.733 $\pm$ 0.014	0.800 $\pm$ 0.012	0.937 $\pm$ 0.006	0.961 $\pm$ 0.009
KaLM-mini	1	0.595 $\pm$ 0.011	0.893 $\pm$ 0.004	0.344 $\pm$ 0.002	0.846 $\pm$ 0.007	0.920 $\pm$ 0.021	0.954 $\pm$ 0.009
	2	0.620 $\pm$ 0.017	0.894 $\pm$ 0.005	0.422 $\pm$ 0.014	0.845 $\pm$ 0.005	0.875 $\pm$ 0.021	0.958 $\pm$ 0.013
	3	0.604 $\pm$ 0.010	0.894 $\pm$ 0.003	0.378 $\pm$ 0.009	0.843 $\pm$ 0.003	0.898 $\pm$ 0.017	0.960 $\pm$ 0.011
	4	0.562 $\pm$ 0.015	0.889 $\pm$ 0.004	0.265 $\pm$ 0.009	0.842 $\pm$ 0.010	0.946 $\pm$ 0.016	0.951 $\pm$ 0.011
	5	0.550 $\pm$ 0.012	0.890 $\pm$ 0.005	0.270 $\pm$ 0.007	0.841 $\pm$ 0.006	0.912 $\pm$ 0.018	0.954 $\pm$ 0.012
	6	0.522 $\pm$ 0.015	0.894 $\pm$ 0.004	0.202 $\pm$ 0.011	0.850 $\pm$ 0.006	0.935 $\pm$ 0.013	0.950 $\pm$ 0.015
	7	0.604 $\pm$ 0.017	0.894 $\pm$ 0.003	0.394 $\pm$ 0.016	0.840 $\pm$ 0.011	0.876 $\pm$ 0.020	0.963 $\pm$ 0.012
	8	0.607 $\pm$ 0.013	0.866 $\pm$ 0.003	0.387 $\pm$ 0.010	0.798 $\pm$ 0.010	0.892 $\pm$ 0.016	0.955 $\pm$ 0.010
	9	0.614 $\pm$ 0.010	0.874 $\pm$ 0.005	0.386 $\pm$ 0.009	0.821 $\pm$ 0.011	0.909 $\pm$ 0.013	0.944 $\pm$ 0.011
	10	0.602 $\pm$ 0.012	0.860 $\pm$ 0.007	0.359 $\pm$ 0.005	0.774 $\pm$ 0.012	0.917 $\pm$ 0.017	0.970 $\pm$ 0.012
	11	0.601 $\pm$ 0.012	0.864 $\pm$ 0.008	0.360 $\pm$ 0.009	0.792 $\pm$ 0.012	0.911 $\pm$ 0.020	0.957 $\pm$ 0.008
	12	0.638 $\pm$ 0.009	0.868 $\pm$ 0.006	0.415 $\pm$ 0.008	0.801 $\pm$ 0.009	0.927 $\pm$ 0.016	0.955 $\pm$ 0.005
	13	0.643 $\pm$ 0.010	0.862 $\pm$ 0.006	0.435 $\pm$ 0.007	0.792 $\pm$ 0.005	0.912 $\pm$ 0.016	0.953 $\pm$ 0.016
	14	0.653 $\pm$ 0.009	0.858 $\pm$ 0.007	0.479 $\pm$ 0.002	0.793 $\pm$ 0.009	0.879 $\pm$ 0.016	0.943 $\pm$ 0.009

(continued on next page)

Task B: Top-K Ranking (5-seed mean $\pm$ std)							
Model	Layer	Acc@1 (base)	Acc@1 (ABTT)	Existing (base)	Existing (ABTT)	New (base)	New (ABTT)
mT5-base	15	0.659 $\pm$ 0.007	0.868 $\pm$ 0.005	0.458 $\pm$ 0.006	0.812 $\pm$ 0.009	0.919 $\pm$ 0.010	0.941 $\pm$ 0.006
	16	0.682 $\pm$ 0.005	0.869 $\pm$ 0.008	0.475 $\pm$ 0.006	0.824 $\pm$ 0.008	0.951 $\pm$ 0.010	0.926 $\pm$ 0.009
	17	0.756 $\pm$ 0.007	0.885 $\pm$ 0.009	0.611 $\pm$ 0.018	0.841 $\pm$ 0.006	0.944 $\pm$ 0.010	0.942 $\pm$ 0.015
	18	0.780 $\pm$ 0.010	0.881 $\pm$ 0.007	0.635 $\pm$ 0.016	0.840 $\pm$ 0.002	0.967 $\pm$ 0.007	0.933 $\pm$ 0.016
	19	0.804 $\pm$ 0.013	0.876 $\pm$ 0.007	0.672 $\pm$ 0.022	0.831 $\pm$ 0.007	0.977 $\pm$ 0.003	0.935 $\pm$ 0.008
	20	0.793 $\pm$ 0.011	0.877 $\pm$ 0.004	0.649 $\pm$ 0.020	0.831 $\pm$ 0.003	0.979 $\pm$ 0.004	0.936 $\pm$ 0.008
	21	0.804 $\pm$ 0.011	0.871 $\pm$ 0.007	0.672 $\pm$ 0.018	0.830 $\pm$ 0.005	0.975 $\pm$ 0.006	0.924 $\pm$ 0.012
	22	0.870 $\pm$ 0.008	0.873 $\pm$ 0.004	0.810 $\pm$ 0.007	0.812 $\pm$ 0.005	0.948 $\pm$ 0.008	0.952 $\pm$ 0.009
	23	0.873 $\pm$ 0.007	0.884 $\pm$ 0.004	0.811 $\pm$ 0.008	0.833 $\pm$ 0.004	0.953 $\pm$ 0.007	0.951 $\pm$ 0.007
	24	0.848 $\pm$ 0.011	0.856 $\pm$ 0.005	0.749 $\pm$ 0.019	0.806 $\pm$ 0.009	0.975 $\pm$ 0.007	0.921 $\pm$ 0.010
	1	0.486 $\pm$ 0.012	0.898 $\pm$ 0.004	0.100 $\pm$ 0.007	0.856 $\pm$ 0.009	0.987 $\pm$ 0.007	0.953 $\pm$ 0.010
	2	0.475 $\pm$ 0.012	0.892 $\pm$ 0.004	0.082 $\pm$ 0.008	0.851 $\pm$ 0.013	0.984 $\pm$ 0.008	0.946 $\pm$ 0.013
	3	0.482 $\pm$ 0.014	0.872 $\pm$ 0.008	0.123 $\pm$ 0.008	0.809 $\pm$ 0.012	0.947 $\pm$ 0.008	0.954 $\pm$ 0.008
	4	0.498 $\pm$ 0.013	0.852 $\pm$ 0.006	0.205 $\pm$ 0.006	0.772 $\pm$ 0.012	0.879 $\pm$ 0.018	0.956 $\pm$ 0.010
	5	0.370 $\pm$ 0.013	0.825 $\pm$ 0.009	0.155 $\pm$ 0.005	0.724 $\pm$ 0.010	0.649 $\pm$ 0.020	0.956 $\pm$ 0.005
	6	0.359 $\pm$ 0.011	0.809 $\pm$ 0.006	0.263 $\pm$ 0.021	0.765 $\pm$ 0.013	0.483 $\pm$ 0.018	0.865 $\pm$ 0.018
	7	0.368 $\pm$ 0.007	0.827 $\pm$ 0.007	0.256 $\pm$ 0.012	0.758 $\pm$ 0.011	0.512 $\pm$ 0.018	0.916 $\pm$ 0.012
	8	0.380 $\pm$ 0.007	0.824 $\pm$ 0.012	0.245 $\pm$ 0.007	0.771 $\pm$ 0.013	0.555 $\pm$ 0.017	0.892 $\pm$ 0.015
	9	0.426 $\pm$ 0.012	0.836 $\pm$ 0.010	0.171 $\pm$ 0.006	0.781 $\pm$ 0.014	0.756 $\pm$ 0.023	0.907 $\pm$ 0.006
	10	0.420 $\pm$ 0.007	0.821 $\pm$ 0.016	0.141 $\pm$ 0.011	0.765 $\pm$ 0.016	0.782 $\pm$ 0.019	0.893 $\pm$ 0.017
	11	0.418 $\pm$ 0.012	0.803 $\pm$ 0.010	0.132 $\pm$ 0.010	0.719 $\pm$ 0.013	0.788 $\pm$ 0.014	0.910 $\pm$ 0.010
	12	0.506 $\pm$ 0.008	0.840 $\pm$ 0.009	0.170 $\pm$ 0.008	0.770 $\pm$ 0.017	0.943 $\pm$ 0.012	0.930 $\pm$ 0.008

Task B operationalises the same embeddings into the deployment workflow, which now requires directory knowledge. We report both routing variants per layer. Table 3 reports the autonomous-routing view: for every test file we compute $s_i = \max_{j \neq i} \cos(e_i, e_j)$ against the rest of the test set and compare to the learned threshold τ , scoring a file correct iff the binary existing-vs-new prediction matches ground truth. Table 4 reports the top- k ranking view in which the system scores each directory by its best document match and presents the top-five options plus the implicit `__NEW__` fallback; the “human” is simulated by selecting the labeled directory whenever it appears in the shortlist, averaged over $M = 5$ query/reference reseeds. Table 5 then reports cumulative top- K accuracy at each model’s best layer for $K = 1, \dots, 5$.

The cumulative view directly answers the deployment question: if the expert inspects the top K candidates, how likely are they to find the correct directory? All six models place the labeled directory in the top-1 position at least 89.4% of the time averaged over five seeds, and by Top-2 every model exceeds 95%. Seed-to-seed standard deviations stay at or below ± 1.1 points, so the ranking of models is stable rather than dependent on the query/reference split. The marginal gains from Top-3 onward are small, suggesting that a shortlist of two or three options suffices for nearly all queries.

4.7 Example-Level Explanations

Integrated gradients are local explanations, so we focus on concrete examples rather than global summaries. For a query passage and one candidate partner passage, we compute per-sequence integrated gradients toward pair cosine similarity and display the original decoded tokens on both axes. Figure 6 gives the main explanation view, a derived pairwise heatmap built from token-token cosine similarity and weighted by the marginal integrated-gradient scores of the two passages.

The pairwise heatmap carries most of the interpretive weight. It is meant to help a human decide whether the retrieved directory looks plausible, not to stand in for a global account of model behavior. Figure 7 adds one more view of the same example.

The attention map provides extra context around the same query and candidate pair. It is not the explanation itself, and it is not meant to make a before-and-after claim about ABTT.

4.8 Retrieval-Aligned Mask Attributions

Figure 8 shows the learned masks on the same PhilTa pair as Figures 6 and 7. Each row masks one side of the pair (query on top, candidate on bottom), and the two columns compare the baseline and ABTT variants of Equation 1. Mask values approach one where the optimizer finds the token essential for preserving the pair cosine, and approach zero where the token contributes little. Padding and tokenizer-artifact positions are frozen at zero by construction.

Baseline and ABTT variants differ in which content positions retain non-trivial mask weight. Because the loss in Equation 1 is trained against the ABTT-cleaned cosine used at retrieval time, the ABTT variant matches the scoring function used in Task A and Task B; we make no empirical claim of superior faithfulness here. Quantitative comparisons against the other attribution methods are reported as accompanying work.

5 Discussion

PCA whitening has worked well for English sentence similarity (Su et al., 2021; Huang et al., 2021), but it performs poorly here. A plausible reason is that whitening removes variance that still helps separate equivalent from non-equivalent Latin legal passages. In this corpus, the original variance structure seems to carry useful signal rather than just noise.

The depth profiles also differ by architecture. The strongest dip appears in the two Latin T5 encoders. mt5-base shows a milder version of the same pattern, and LaBSE plus the decoder-only models vary much less across depth. Architecture and pre-training objective are the most likely reasons. After ABTT, those differences matter less in practice because the weak middle layers become much more usable. It also reduces the need to hunt for a narrow “good” layer band before retrieval.

We also tested SIF-weighted pooling and whitening as simpler alternatives. They can help in individual cases, but neither produces the same consistent repair across models and layers. Mean pooling plus ABTT remains the safer default for this retrieval pipeline.

The explanation component is intentionally local. The derived pair matrix and the companion attention map are there to help a scholar inspect one ranked option, not to summarize the model as a whole. That fits the human-in-the-loop directory-

Model	Top-1	Top-2	Top-3	Top-4	Top-5
LaTa	90.1 $\pm$ 1.1	96.0 $\pm$ 0.4	97.1 $\pm$ 0.1	97.6 $\pm$ 0.4	97.7 $\pm$ 0.4
PhilTa	90.0 $\pm$ 0.9	95.7 $\pm$ 0.4	96.9 $\pm$ 0.4	97.4 $\pm$ 0.3	97.7 $\pm$ 0.2
LaBSE	90.4 $\pm$ 0.8	95.5 $\pm$ 0.4	96.4 $\pm$ 0.3	97.1 $\pm$ 0.5	97.7 $\pm$ 0.4
Qwen3-0.6B	90.3 $\pm$ 0.6	95.5 $\pm$ 0.5	96.8 $\pm$ 0.3	97.2 $\pm$ 0.5	97.4 $\pm$ 0.4
KaLM-mini	89.4 $\pm$ 0.5	95.1 $\pm$ 0.4	96.4 $\pm$ 0.2	96.9 $\pm$ 0.3	97.3 $\pm$ 0.4
mt5-base	89.8 $\pm$ 0.4	95.6 $\pm$ 0.5	96.6 $\pm$ 0.3	96.9 $\pm$ 0.2	97.2 $\pm$ 0.2

Table 5: Cumulative top- K accuracy for Task B across the six models, reported as mean $\pm$ standard deviation over $M = 5$ query/reference reseeds at a fixed v2 train/test split. Each cell gives the percentage of test queries whose labeled directory appears within the model’s top K options at its best layer/method configuration. All models exceed 89% by Top-1 and 95% by Top-2. Only ABTT post-processing is reported in the main paper; SIF-only, PCA whitening, and SIF+ABTT are reported in the appendix.



Figure 6: Derived pairwise heatmap for one PhilTa retrieval case. The left panel shows the baseline representation and the right panel shows the same example after ABTT. Both axes use original decoded tokens, and the cell values combine token-token cosine similarity with single-sequence integrated-gradient scores.

fig_attention_philta.pdf

Figure 7: Companion self-attention maps for the same PhilTa example. Because ABTT is applied after encoding, attention is included only as supporting context, not as a before-and-after comparison.

assignment setting much better than our earlier aggregate token-category analyses. We also ran the before-and-after ABTT comparison using five additional attribution methods beyond integrated gradients on the same example pairs, with consistent improvements in attribution clarity across all six models; for brevity we report only the IG pair-matrix view in the main text.

Limitations

Our study covers one domain, medieval canon law, and one language, Latin. The 1,705-fragment corpus is large for this field, but it is still small by current NLP standards. We test six models, and mod-

els outside this set may behave differently. ABTT also needs enough unlabeled documents to estimate principal components well, and we have not tested it in settings with only a few hundred texts or fewer. The explanation analysis is local and example-based, and the human-in-the-loop evaluation is a simulation: we assume the labelled directory would be chosen whenever it appears in the model’s shortlist, which captures ranking quality but not the judgment load of a working scholar.

Our example-level attributions use integrated gradients, with a retrieval-aligned learned-mask variant adapted from MaRC (Brinner and Zarri , 2023) trained directly against the ABTT-cleaned

fig_retrieval_mask_pair_philta.pdf

Figure 8: Retrieval-aligned mask attributions for the same PhilTa pair as Figure 6. Top row: mask on the query with the candidate held fixed. Bottom row: mask on the candidate with the query held fixed. Left column: baseline cosine target. Right column: ABTT-cleaned cosine target. Token labels are the original decoded tokens; frozen non-content positions are shown in gray.

cosine used for ranking as a complementary view. Both remain local explanations and do not summarize model behaviour globally, and both still depend on the choice of layer at which the hidden state is read. The joint two-sided optimization, in which a single mask is fit over both passages simultaneously rather than one side at a time, is left to future work, as is the quantitative faithfulness evaluation across attribution methods.

References

- Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. [A simple but tough-to-beat baseline for sentence embeddings](#). In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- Marc Brinner and Sina Zarrieß. 2023. Model interpretability and rationale extraction by input mask optimization. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13722–13744. Association for Computational Linguistics.
- Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. 2022. [Language-agnostic BERT sentence embedding](#). In *Proceedings of the*

60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 878–891. Association for Computational Linguistics.

Junjie Huang, Duyu Tang, Wanjun Zhong, Shuai Lu, Linjun Shou, Ming Gong, Daxin Jiang, and Nan Duan. 2021. [WhiteningBERT: An easy unsupervised sentence embedding approach](#). In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 238–244. Association for Computational Linguistics.

Xinyu Lu, Nurlan Maharramov, and 1 others. 2025. [KaLM-embedding: Superior training data brings a stronger embedding model](#). Accessed: 2025.

Leland McInnes, John Healy, and James Melville. 2018. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.

Jiaqi Mu and Pramod Viswanath. 2018. [All-but-the-top: Simple and effective postprocessing for word representations](#). In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

Qwen Team. 2025. [Qwen3-embedding: Advancing text embedding and reranking through foundation models](#). Accessed: 2025.

Frederick Riemenschneider and Anette Frank. 2023. [Exploring large language models for classical philology](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15181–15199. Association for Computational Linguistics.

Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. 2021. [Whitening sentence representations for better semantics and faster retrieval](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328. PMLR.

Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. [mT5: A massively multilingual pre-trained text-to-text transformer](#). In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498. Association for Computational Linguistics.

A Last-Token vs. Mean Pooling

A natural question for the decoder-only models (Qwen3-0.6B, KaLM-mini, Qwen3-8B) is whether last-token pooling, the canonical choice for this family, would change our picture. To answer it directly, we re-run the ABTT post-processing pipeline on all six models using last-token pooling in place of mean pooling, keeping every other hyperparameter and the same train/test split fixed. We restrict the comparison to the two ABTT variants (fixed $D=10$ and per-layer tuned D) that carry the main-text analysis; SIF weighting is not defined for single-token pooling and is therefore omitted. Table 6 reports, for each (model, pooling) pair, the layer that maximizes Task B autonomous-routing assignment accuracy together with Task B directory accuracy at rank 1 and the cosine gap at that same layer. Mean pooling outperforms last-token pooling on assignment accuracy for every model; the gap is largest on the two Latin T5 encoders (LaTa and PhilTa, where last-token pooling loses roughly 23–31 accuracy points) and narrows to 1–5 points on LaBSE and the decoder-only models (Qwen3-0.6B, KaLM-mini, Qwen3-8B), confirming that mean pooling is the safer default for this retrieval pipeline even on the decoder-only architectures where last-token pooling is the more conventional choice.

B Cluster Geometry: Additional Models

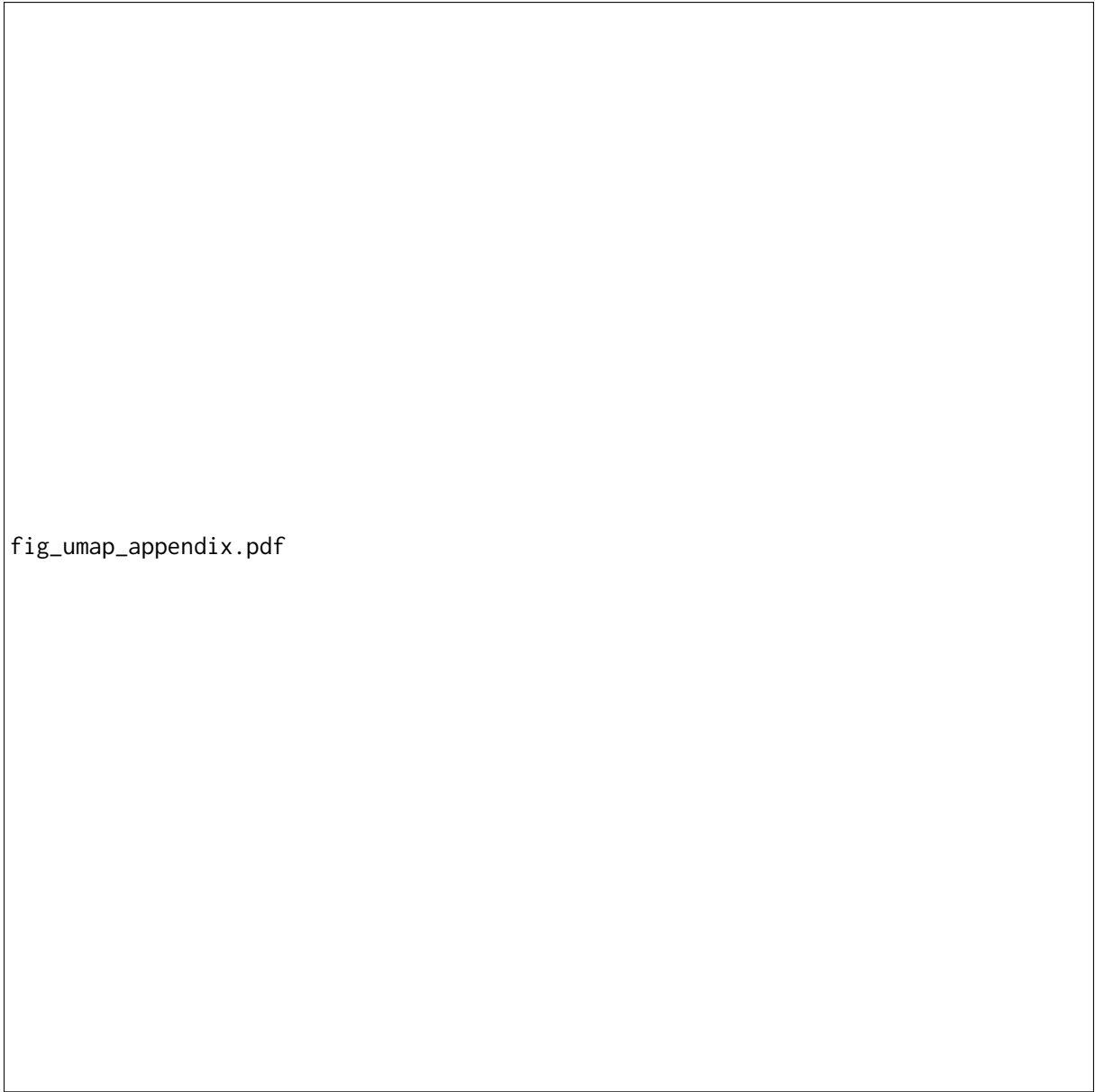
Figures 9 and 10 extend the cluster-geometry panels of Figures 4 and 5 to the remaining three models (LaBSE, Qwen3-0.6B, KaLM-mini), using the same six hand-picked directories and the same baseline vs. pure ABTT ($D=10$) comparison. The pattern observed in the main paper persists: under ABTT, winnable-fragment silhouette rises by 0.13, 0.66, and 0.42 on t-SNE for the three models respectively (0.16, 0.56, 0.37 on UMAP), and colored points collapse into tighter per-directory groups. The largest effect is on Qwen3-0.6B, whose baseline silhouette is negative on both projections and becomes strongly positive after ABTT.

Table 6: Mean vs. last-token pooling on the balanced split, restricted to ABTT post-processing. Best layer per (model, pooling) selected by Task A assignment accuracy. Task B metrics (Acc@1, cosine gap) reported at that same layer.

Model	Pool	Layer	Method	Assign Acc	Acc@1	Gap
LaTa	Mean	1	ABTT (D=10)	0.909	0.880	0.565
LaTa	LastTok	12	ABTT (D=10)	0.676	0.425	0.276
PhilTa	Mean	1	ABTT (D=10)	0.911	0.883	0.573
PhilTa	LastTok	9	ABTT (D=10)	0.604	0.385	0.200
LaBSE	Mean	11	ABTT (D=10)	0.908	0.885	0.610
LaBSE	LastTok	12	ABTT (D=10)	0.895	0.871	0.586
Qwen3-0.6B	Mean	5	ABTT (D=10)	0.915	0.894	0.545
Qwen3-0.6B	LastTok	27	ABTT (D=10)	0.907	0.885	0.562
KaLM-mini	Mean	1	ABTT (D=10)	0.921	0.902	0.568
KaLM-mini	LastTok	24	ABTT (D=10)	0.875	0.847	0.483
Qwen3-8B	Mean	1	ABTT (D=10)	0.924	0.897	0.543
Qwen3-8B	LastTok	35	ABTT (D=10)	0.910	0.892	0.642

fig_tsne_appendix.pdf

Figure 9: t-SNE projections of the six most-populated directories at each model’s best layer for LaBSE, Qwen3-0.6B, and KaLM-mini. Same directory palette as Figure 4; baseline (left) vs. pure ABTT ($D=10$) (right). ABTT tightens same-color clusters for every model.



fig_umap_appendix.pdf

Figure 10: UMAP projections matching Figure 9. The cluster-tightening is reproduced under a different projection algorithm.